Data mining & machine learning

2. Outlier Detection

Francesco Pistolesi
University of Pisa
francesco.pistolesi@unipi.it

M.Sc. in Artificial Intelligence and Data Engineering

UNIVERSITÀ DI PISA

Learning goals

By the end of the lecture, you will be able to:

- define what an **outlier** is and why detecting it matters;
- understand how **DBSCAN** detects outliers as noise points;
- explain and use the k-**dist plot** to set the ε parameter;
- compute and interpret the **Local Outlier Factor (LOF)** score;
- tune the **contamination** parameter and understand its effect on LOF.

Main topics

outlier definition

DBSCAN as detector

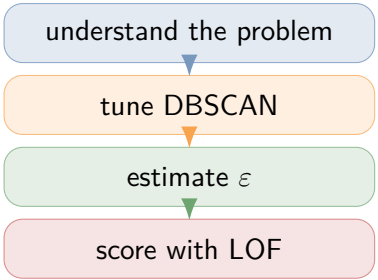
k-dist heuristic

LOF score

Roadmap

Notebook structure

1. What is an outlier?
2. Dataset construction
3. Outlier detection with DBSCAN
4. Choosing ε: the k-dist plot
5. Outlier detection with LOF
6. Interpreting the LOF score

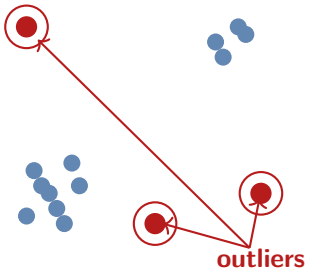


What is an outlier?

An **outlier** (or *anomaly*) is an observation that significantly deviates from the others, arousing suspicion of being generated by a different mechanism.

Outliers arise from many sources:

- measurement or recording **errors**;
- rare but **genuine phenomena** (fraud, faults, novel events);
- objects from a **different distribution**.



Key distinction

Whether an outlier is *noise to remove* or *signal to detect* entirely depends on the application.

A taxonomy of anomaly detection approaches

Approach	Idea	Examples
Statistical	model the normal distribution; flag low-probability points	Z-score, Grubbs test
Distance-based	outliers are far from their neighbors	k-NN distance
Density-based	outliers live in low-density regions	**DBSCAN**, **LOF**
Model-based	outliers are hard to reconstruct	Isolation Forest, Autoencoders

unsupervised setting

no labels required

The notebook's synthetic dataset

Construction

- **1000** samples from 5 Gaussian blobs
- Four blobs have $\sigma = 0.1$ (tight)
- One blob has $\sigma = 1.0$ (sparse)
- **20 uniform outliers** added in $[-10, 10]^2$

This design is deliberate:

- the sparse blob **challenges** density-based methods;
- the uniform noise provides a **ground truth** for evaluation.

```
from sklearn.datasets import make_blobs

X, y = make_blobs(
    n_samples=1000,
    centers=5,
    cluster_std=[0.1,0.1,0.1,0.1,1],
    center_box=(-10,10),
    n_features=2,
    random_state=23)

outliers = np.random.uniform
    (-10,10,(20,2))
data = np.concatenate((X, outliers))
target = np.concatenate(
    (y, [-1]*len(outliers)))
target_anomaly = np.minimum(target,0)
    *(-1)
```

Evaluation in an unsupervised setting

In real applications the **ground truth is unknown**.

In the notebook, ground truth is available because the dataset is synthetic. We encode it as follows:

$$\texttt{target\_anomaly}_i = \begin{cases} 1 & \text{if point } i \text{ is a true outlier} \\ 0 & \text{otherwise} \end{cases}$$

We then treat the detector's output as a **binary classifier** and compare it to the ground truth using a classification report.

Handle with care!

This evaluation is only possible here **because we *purposely constructed* the data**. In practice, precision and recall on outliers cannot be computed without expert labelling.

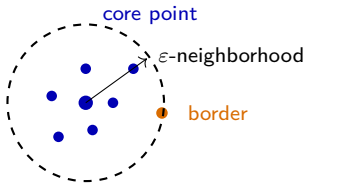
DBSCAN: density-based clustering

DBSCAN identifies clusters as connected dense regions, and labels isolated points as noise. It uses two parameters:

- ε: radius of the local neighborhood
- *MinPts*: minimum number of points required to define a dense region

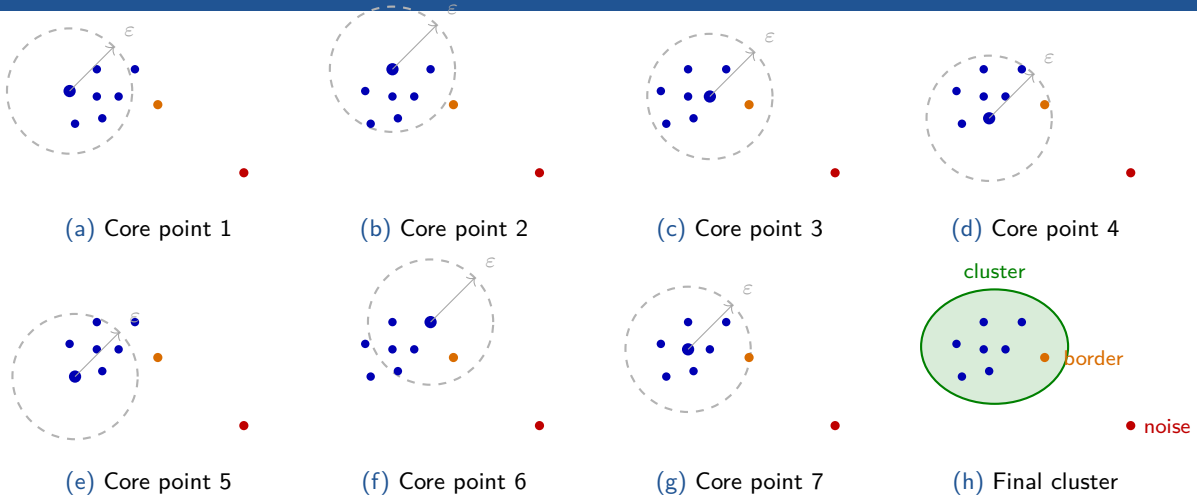
Each point is classified as:

- **Core point**: has at least *MinPts* points in its ε-neighborhood
- **Border point**: lies in the neighborhood of a core point, but does not satisfy the core condition
- **Noise point**: does not belong to any dense region



MinPts points inside the dashed circle

Key idea: DBSCAN grows a cluster by connecting nearby core points and then assigning border points to that cluster.



(a) Core point 1 (b) Core point 2 (c) Core point 3 (d) Core point 4

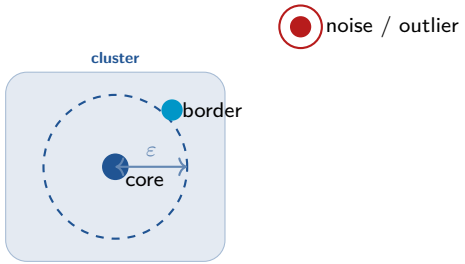
(e) Core point 5 (f) Core point 6 (g) Core point 7 (h) Final cluster

Figure: DBSCAN: ε-neighborhood from each core point. The last figure shows the resulting cluster (green), obtained by connecting density-reachable core points and including the border point (orange).

DBSCAN: density-based clustering recap

DBSCAN (Density-Based Spatial Clustering of Applications with Noise) partitions points into:

- **Core points**: have at least *MinPts* neighbors within distance ε
- **Border points**: reachable from a core point, but not core themselves
- **Noise points**: neither core nor border; labeled -1



Key insight for outlier detection

Noise points are DBSCAN's **natural outliers**: they do not belong to any cluster.

DBSCAN parameters

ε (eps)

Radius of the neighborhood. Controls the spatial scale at which density is measured.

- Too small $\Rightarrow$ many noise points (low precision)
- Too large $\Rightarrow$ outliers absorbed into clusters (low recall)

MinPts

Minimum number of points inside the ε-neighborhood for a point to be a core point.

- Common heuristic: $MinPts \geq D + 1$ (where D = number of features).
- In the notebook: $MinPts = 4$.

ε controls scale

MinPts controls density threshold

Note: remember that *points* may also be referred to as *samples*.

DBSCAN in scikit-learn

```
from sklearn.cluster import DBSCAN
from sklearn.metrics import classification_report

min_samples = 4
for eps in [0.2, 0.5, 1, 2]:
    dbscan = DBSCAN(
        min_samples=min_samples,
        eps=eps)
    y_assignment = dbscan.fit_predict(data)

    n_cluster = len(np.unique(y_assignment)) - 1
    n_outlier = list(y_assignment).count(-1)

    # DBSCAN labels noise as -1
    # convert: -1 -> 1 (outlier), else -> 0
    y_eval = np.minimum(y_assignment, 0) * (-1)
    print(classification_report(
        target_anomaly, y_eval))
```

Reading the output

- `fit_predict` returns cluster ids
- label $= -1$ means **noise/outlier**
- We remap: $-1 \rightarrow 1$, rest $\rightarrow 0$ for classification metrics

Effect of ε on DBSCAN outlier detection

ε	Clusters	Points labeled noise	Precision	Recall
0.2	many small	very high	very low	very high
0.5	reasonable	moderate	moderate	high
1.0	fewer, larger	low	higher	lower
2.0	very few	very low	high	very low

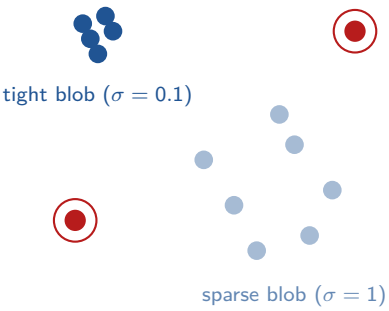
Trade-off

As ε increases, DBSCAN labels fewer normal points as outliers (**precision is increased**), but some true outliers are absorbed into nearby clusters (**recall is decreased**).

The sparse blob problem

One of the five blobs has standard deviation $\sigma = 1.0$, much larger than the others.

- With small ε (e.g. 0.2), points in the sparse blob are **not dense enough** to form a cluster: DBSCAN marks them as noise.
- This dramatically inflates the false-positive count.
- Recall is high (all true outliers are caught), but **precision collapses** to ≈ 0.16.



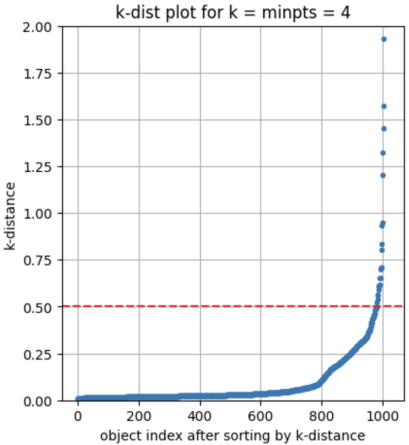
Limitation

DBSCAN uses a **global** density threshold: it struggles when data contains clusters of **varying levels of density**.

How to choose ε: the k-dist plot

The k-**dist plot** is a heuristic for estimating a good ε:

1. Fix $k = MinPts$.
2. For each point, compute the distance to its k-th nearest neighbor.
3. Sort distances in ascending order and plot.
4. Identify the **"elbow"**: the point of maximum curvature.
5. Set ε at the elbow value.



Intuition

Points with a small k-distance are in dense regions (inliers). The elbow separates them from sparser, outlier-like points.

k-dist plot in scikit-learn

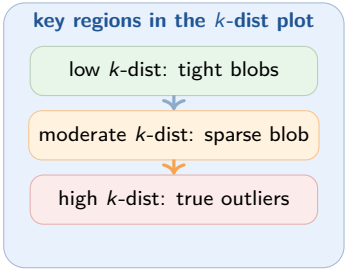
```python
from sklearn.neighbors import NearestNeighbors

minpts = 4
nbrs = NearestNeighbors(
    n_neighbors=minpts).fit(data)
distances, indices = nbrs.kneighbors(data)

# distance to the k-th nearest neighbor
k_dist = [x[-1] for x in distances]

# sort and plot
plt.plot(sorted(k_dist), '.')
plt.axhline(0.5, color='r', linestyle='--')
plt.xlabel('object index after sorting')
plt.ylabel('k-distance')
plt.ylim([0, 2])
```

Notebook observation

- Sharp rise around index 1000 = true outliers
- Smaller step at ≈ 800 = sparse blob
- Red dashed line at 0.5 marks the chosen ε

Reading the k-dist plot: two elbows

- The **leftmost** region (low k-distance) corresponds to points inside the four tight blobs.
- A **moderate step** around index 800 separates the sparse blob.
- A **sharp jump** near index 1000 marks the 20 true outliers.



Practical challenge

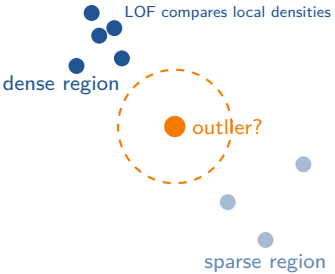
In this dataset there are *two* elbows. Identifying the right one requires domain knowledge or experimentation. This is a fundamental limitation of the k-dist heuristic.

Local Outlier Factor: motivation

DBSCAN uses a **global** density threshold ε. This is problematic when:

- clusters have **different densities**;
- there is no single ε that works everywhere.

LOF addresses this by comparing each point's local density to that of its neighbors.



Core idea

A point is an outlier if its neighborhood is **significantly less dense** than the neighborhoods of its own neighbors.

LOF: key definitions

Let k be a fixed neighborhood size and $N_k(p)$ be the k nearest neighbors of p.

Reachability distance

$$\text{reach-dist}_k(p, o) = \max(k\text{-dist}(o),\ d(p, o))$$

A smoothed distance from p to o: never smaller than o's own k-distance.

Local reachability density (LRD)

$$\text{lrd}_k(p) = \left(\frac{\sum_{o \in N_k(p)} \text{reach-dist}_k(p, o)}{|N_k(p)|} \right)^{-1}$$

Inverse of the average reachability distance to p's k neighbors.

Local Outlier Factor

$$\text{LOF}_k(p) = \frac{1}{|N_k(p)|} \sum_{o \in N_k(p)} \frac{\text{lrd}_k(o)}{\text{lrd}_k(p)}$$

Average ratio of neighbors' LRD to p's LRD.

- $\text{LOF}(p) \approx 1 \Rightarrow$ **inlier**: p has similar density as its neighbors.
- $\text{LOF}(p) \gg 1 \Rightarrow$ **outlier**: p's neighborhood is much less dense.

LOF in scikit-learn

```
from sklearn.neighbors import LocalOutlierFactor as
    LOF

for contamination in [0.01, 0.02, 0.05, 'auto']:
    lof = LOF(
        n_neighbors=10,
        contamination=contamination)
    y_lof = lof.fit_predict(data)

    # LOF: +1 = inlier, -1 = outlier
    # remap to: 0 = normal, 1 = outlier
    y_eval = np.minimum(y_lof, 0) * (-1)
    print(classification_report(
        target_anomaly, y_eval,
        target_names=['normal','outlier']))
```

API notes

- `n_neighbors` $= k$
- Labels: $+1$ inlier, -1 outlier
- `contamination` controls the decision threshold

The contamination parameter

`contamination` is the **expected fraction of outliers** in the data.

It sets the decision threshold (`offset_`) on the LOF score:

How the offset is computed

- `'auto'`: `offset_` $= -1.5$
- numeric value q:
  `offset_` = the q-th percentile of `negative_outlier_factor_`

A point is flagged as outlier if its negative LOF $<$ `offset_`.

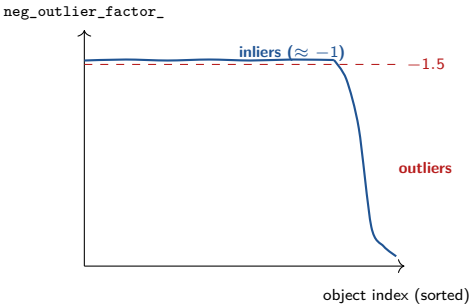
contamination	Prec.	Rec.
0.01	high	low
0.02	**good**	**good**
0.05	lower	higher
`'auto'`	moderate	moderate

True outlier ratio $= \frac{20}{1020} \approx 0.02$
Setting contamination $\approx$ true ratio gives the best results.

The `negative_outlier_factor_` attribute

After fitting, LOF exposes `negative_outlier_factor_`:

- It is the **opposite** of LOF: value $= -\text{LOF}(p)$.
- The **higher** (less negative), the more normal.
- Inliers: values close to -1.
- Outliers: large negative values (e.g. -10, -50, ...).



Why negative?

Scikit-learn convention: *higher scores are better*.
Negating LOF follows this convention.

Visualizing LOF scores on the 2D space

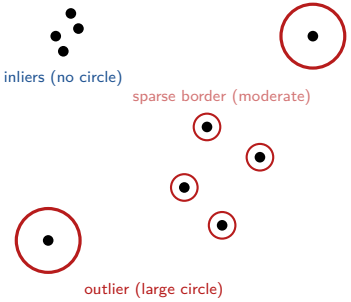
```
X_scores = lof.negative_outlier_factor_

plt.scatter(data[:, 0], data[:, 1],
            color='k', s=3.0,
            label='Data points')

# circles with radius proportional to outlier score
radius = (X_scores.max() - X_scores) \
       / (X_scores.max() - X_scores.min())

plt.scatter(data[:, 0], data[:, 1],
            s=1000 * radius,
            edgecolors='r',
            facecolors='none',
            label='Outlier scores')
plt.title('Local Outlier Factor (LOF)')
```

How to read it

- Large red circles = high LOF = strong outliers
- Small or absent circles = inliers
- Cluster-edge points may get moderate scores

Interpreting the LOF visualization

- The 2 uniform-noise points get **very large** circles: they have much lower local density than their neighbors.
- Points at the **border of the sparse blob** may receive LOF slightly above 1 (may be treated as weak outliers) based on how their local densities compare to those of their neighbors.
- Points inside tight blobs get LOF ≈ 1: **very small or no visible circle**.



LOF advantage over DBSCAN

LOF produces a **continuous score**, not just a binary label. This allows finer-grained analysis, ranking, and threshold sensitivity analysis.

DBSCAN vs LOF: a comparison

Aspect	DBSCAN	LOF
Output	cluster labels + noise	continuous outlier score
Density model	global (ε, *MinPts*)	local (per-point)
Mixed-density data	struggles	handles well
Key parameters	ε, *MinPts*	k, contamination
Parameter tuning	k-dist plot for ε	contamination $\approx$ true fraction
Interpretability	binary (in/out cluster)	graded score
Scalability	fast ($O(n \log n)$)	slower ($O(n^2)$ naive)

DBSCAN: clustering + detection

LOF: scoring & ranking

Best practices for outlier detection

1. **Inspect the data first.** Scatter plots and k-dist plots reveal structure before any algorithm runs.
2. **Set *MinPts* / k from dimension.** Common rule: $\mathit{MinPts} \geq D + 1$.
3. **Use the k-dist plot** to choose ε for DBSCAN.
4. **Set contamination from domain knowledge.** If the expected outlier rate is known, use it; otherwise start with `'auto'`.
5. **Evaluate cautiously.** Ground truth is rarely available; use precision/recall only when labels exist.
6. **Use LOF scores, not just labels.** The continuous score enables ranking and sensitivity analysis.

Further reading

- Breunig, M. M., Kriegel, H. P., Ng, R. T., Sander, J. (2000). *LOF: Identifying Density-Based Local Outliers*. SIGMOD.
- Ester, M., Kriegel, H. P., Sander, J., Xu, X. (1996). *A Density-Based Algorithm for Discovering Clusters in Large Spatial Databases with Noise*. KDD.
- Chandola, V., Banerjee, A., Kumar, V. (2009). *Anomaly Detection: A Survey*. ACM Computing Surveys.
- Goldstein, M. and Uchida, S. (2016). *A Comparative Evaluation of Unsupervised Anomaly Detection Algorithms for Multivariate Data*. PLOS ONE.
- Scikit-learn documentation: `sklearn.cluster.DBSCAN`, `sklearn.neighbors.LocalOutlierFactor`.

Conclusions

Core ideas

- Outliers are points that deviate from the norm — noise or signal depending on the context.
- DBSCAN detects outliers as noise, but uses a **global** density scale.
- The k-dist plot helps choose ε via the elbow heuristic.

Methodological message

- LOF assigns a **local, continuous** score that handles variable-density data better.
- The contamination parameter controls the decision boundary.
- No method wins universally: choose based on data structure and the application.

understand the data | tune carefully | evaluate honestly

Questions?